
高级语言程序设计

实 验 报 告

南开大学

计算机大类

姓名：吴昀青

学号：2512318

班级：模拟 3-2 班

2026 年 5 月 5 日

目录

1	作业题目	2
2	开发软件	2
3	课题要求	2
4	主要流程	3
4.1	模拟水果抛射与波次控制	3
4.2	刀光的物理模拟与平滑处理	3
4.3	切割效果与连击判定	4
5	收获	4

1 作业题目

基于 EasyX 的平面动作游戏——水果忍者
(Fruit Ninja)

2 开发软件

- Visual Studio 2022
- EasyX 图形库
- C++17 标准

3 课题要求

1. 面向对象
2. 使用 C++
3. 实现图形化

4 主要流程

4.1 模拟水果抛射与波次控制

封装 Fruit 类模拟水果的抛射过程。首先初始化水果的位置和速度（通过 difficultySpeedMultiplier，随着分数增加，水果的初始速度会获得 1.01x 至 1.03x 的微小倍率加成），同时增加角速度让水果实现自旋。其次利用 ScatterGroup 和 VerticalCombo 实现水果的独立生成和成排批量生成，将炸弹设计成 Fruit 的实例，与水果共享同一套重力、位移和回收逻辑，在渲染层通过 isBomb() 钩子触发特殊的烟雾效果。

在实际实现过程中，主要遇到了两个问题：

其一为水果在边缘位置的处理。通过 biasVelocityInward 函数进行内偏导向：如果水果在左侧边缘（< 210.0）生成，强制修正其 vx 为正数，右侧反之。这确保了即便生成位置偏僻，水果也会向屏幕中心汇聚，减少了水果直接飞出侧边导致无法被切到。

其二为切水果时的碰撞检测。将判定半径乘以 expandScale（约 1.15）外加 10 像素余量，避免快速划动时带来的误差。同时采用点到线段的距离平方检测（distanceSqPointToSegment），即便玩家划屏极快，两帧之间的刀光路径依然能精确捕获路径上的所有水果，防止漏检。使用距离平方而非实际距离进行比较，避免了开方运算的性能开销。

封装 WaveController 类运用波次控制水果的出现，建立 buildGroup 函数对应十组波次类型，其中前三组为出现顺序固定来示意游戏逻辑，后七组引入 kWeightedGroups 数组，给不同类型的波次分配了权重，定义 advanceGroup，根据组索引调用 buildGroup 函数实现水果出现的波次切换。在实际运行中为了避免水果出现过于单调，在不同的波次设置多种类型如 CalmSingle、VerticalCombo、MixedRush 分别实现单个抛出、垂直多个出现，实现连击效果（引入了 useSameFruitVariant 使得该波次出现的所有水果均为同一种），以及水果与炸弹同时出现。当一个波次组结束时，先调用 advanceGroup 通过权重随机选择下一个组索引，随后调用 primeGroup 重置组内计时器并预热该组的第一波水果抛出。FruitField 通过 tryConsumeBatch 消费 batch，根据 batch 参数调用 spawnOneBomb、spawnVerticalGroup 或 spawnScatterGroup 完成实际生成。

4.2 刀光的物理模拟与平滑处理

通过 FruitKnife 类封装玩家挥砍的交互逻辑。系统实时记录鼠标位移路径上的采样点并封装为 TrailPoint 结构，存入双端队列 std::deque 中。每个轨迹点均带有 life 生命周期（约 0.25 秒），随时间线性衰减，衰减至 0 后由 update() 函数自动移除，从而实现刀光逐渐缩短的效果。

在实际实现中，重点解决了两个问题：

其一为轨迹的平滑度处理。若直接连接鼠标原始点，在高速移动下会产生明显的折线感。为此，通过 buildSmoothedTrail() 函数对原始采样点进行平滑插值处理，消除

锯齿感，使刀光更接近弧形条带。

其二为刀光的视觉与听觉反馈。通过 `KnifeEffect` 类渲染带有半透明效果的竹叶风格 (Bamboo Leaves) 采样图片，并利用 `Math::atan2` 计算轨迹切线方向，通过 `drawImageAlphaRotated` 让贴图始终沿挥砍方向偏转。同时，在 `update()` 中计算相邻点的位移与 `dt` 的比值以获取瞬时速度，当速度超过 `kKnifeMinSpeedPxPerSec` 阈值时，由 `MusicController` 触发 `playSfx(SfxId::Knife1~4)`，确保慢速划动时不产生噪音，仅在快速暴击时提供破空声。

4.3 切割效果与连击判定

当 `trySliceAlongSegment` 检测到刀光线段穿过水果时，根据切割角度 `bladeAngle` 瞬时生成两个 `FruitHalf` 对象，并由 `inheritX/Y` 继承原果实的运动惯性。通过 `splitPush` 为两半果实施加背离切割线的反冲力，并分别赋予方向相反的额外角速度 `angularVelocity`，模拟出果实被劈开后向两侧飞旋而下的物理效果。

其次为瞬间视觉冲击设计。在切割点 `impactPoint`，系统通过 `makeFlashEffect` 绘制一个短生存期的 `flashImage`，模拟爆裂闪光；同时调用 `makeJuiceEffect` 根据果实种类提取 `juiceColor`，向四周喷射具有重力感应的粒子。所有残骸与粒子均共享 `isOffScreen()` 检测逻辑，由 `std::remove_if` 在飞出视野后自动回收。

封装 `ComboJudge` 类实现连击效果，利用定义的 `kComboWindowSec` (0.5 秒) 时间窗口，通过 `registerHit()` 将每次成功的切割动作压入 `hits_` 时间戳队列。当队列中处于窗口内的有效切割次数 ≥ 2 时，由 `update()` 结算函数判定为连击生效。

5 收获

- 感受到封装类的重要性：将水果、刀光、特效、音效等封装为独立类 (`Fruit`、`Knife`、`Effect`、`MusicController` 等)，通过组合与回调机制，代码结构清晰，便于维护和扩展。
- 学习了碰撞检测优化：使用距离平方比较避免开方运算，在大量水果同时存在时保持了流畅的帧率。
- 学习了事件驱动音效系统：通过场景切换自动切换 BGM，通过事件触发音效播放，配合独立线程处理音效队列，避免主线程阻塞。
- 学习了状态机模式：通过场景状态机 (`Entrance` $\rightarrow$ `Idle` $\rightarrow$ `StartGame/ExitGame`) 管理游戏流程，使各阶段逻辑清晰分离，炸弹爆炸后的 `BombTransitionPhase` 设计确保了平滑过渡。

AI 工具

在本项目开发过程中，使用了以下 AI 工具：

1. 使用 DeepSeek 实现对波次控制的初步设计，以及对水果抛射转动的优化；
2. 使用 GPT-5.4 实现对刀光效果的平滑处理以及对连击的判断。